

WEEK 7 ASSIGNMENT

Uploaded the Dataset “*Sample - Superstore*” in databricks software where the .CSV dataset is converted to Delta Table where different operations such as merge , delete() , update can be performed.

The screenshot shows the Databricks 'Create or modify table from file upload' interface. The file 'Sample - Superstore.csv' (2.29MB) is uploaded. The table name is 'sample_superstore'. The preview shows 15 rows of data with columns: Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, and Count.

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Count
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United State
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United State
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United State
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United State
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United State
6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United State
7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United State
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United State
9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United State
10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United State
11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United State
12	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United State
13	CA-2017-114412	2017-04-15	2017-04-20	Standard Class	AA-10480	Andrew Allen	Consumer	United State
14	CA-2016-161389	2016-12-05	2016-12-10	Standard Class	IM-15070	Irene Maddox	Consumer	United State
15	US-2015-118983	2015-11-22	2015-11-26	Standard Class	HP-14815	Harold Pawlan	Home Office	United State

STEP 1: Load dataset into a Delta table.

The screenshot shows the Databricks SQL Editor with the command: `df = spark.table('workspace.default.sample_superstore')`. The preview shows 15 rows of data with columns: Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, Country, City, and State.

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kent
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kent
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	Calif.
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Floric
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Floric
6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif.
7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif.
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif.
9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif.
10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif.
11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif.
12	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif.
13	CA-2017-114412	2017-04-15	2017-04-20	Standard Class	AA-10480	Andrew Allen	Consumer	United States	Concord	Norti
14	CA-2016-161389	2016-12-05	2016-12-10	Standard Class	IM-15070	Irene Maddox	Consumer	United States	Seattle	Wash

STEP 2: Perform basic cleaning (handle nulls, remove duplicates).

CLEAN DATA

```
clean_df = df.dropna() # drop null values
clean_df = clean_df.dropDuplicates() # drop duplicates
display(clean_df)
```

> See performance (1)

> clean_df: pyspark.sql.connect.dataframe.DataFrame = [Row ID: long, Order ID: string ... 19 more fields]

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	
1	423	CA-2017-125388	2017-10-19	2017-10-23	Standard Class	MP-17965	Michael Paige	Corporate	United States	Lawrence	Massachusetts
2	443	CA-2016-115756	2016-09-05	2016-09-07	Second Class	PK-19075	Pete Kriz	Consumer	United States	Detroit	Michigan
3	446	CA-2016-115756	2016-09-05	2016-09-07	Second Class	PK-19075	Pete Kriz	Consumer	United States	Detroit	Michigan
4	455	CA-2017-147277	2017-10-20	2017-10-24	Standard Class	EB-13705	Ed Braxton	Corporate	United States	Akron	Ohio
5	456	CA-2016-100153	2016-12-13	2016-12-17	Standard Class	KH-16630	Ken Heidel	Corporate	United States	Norman	Oklahoma
6	465	CA-2016-109869	2016-04-22	2016-04-29	Standard Class	TN-21040	Tanja Norvell	Home Office	United States	Phoenix	Arizona
7	477	CA-2014-144666	2014-11-09	2014-11-11	Second Class	JP-15520	Jeremy Pistek	Consumer	United States	San Francisco	California
8	498	CA-2016-126158	2016-07-25	2016-07-31	Standard Class	SC-20095	Sanjit Chand	Consumer	United States	Costa Mesa	California
9	508	CA-2015-145352	2015-03-16	2015-03-22	Standard Class	CM-12385	Christopher Martinez	Consumer	United States	Atlanta	Georgia
10	520	CA-2015-157812	2015-03-22	2015-03-26	Standard Class	DB-13210	Dean Braden	Consumer	United States	Houston	Texas
11	523	CA-2017-145142	2017-01-23	2017-01-25	First Class	MC-17605	Matt Connell	Corporate	United States	Detroit	Michigan
12	540	CA-2015-134894	2015-12-07	2015-12-11	Standard Class	DK-12985	Darren Koutras	Consumer	United States	Henderson	Kentucky
13	550	CA-2015-113173	2015-11-15	2015-11-17	Second Class	DK-13225	Dean Katz	Corporate	United States	Chicago	Illinois
14	562	CA-2014-145576	2014-09-14	2014-09-18	Second Class	CA-12775	Cynthia Arntzen	Consumer	United States	Tampa	Florida

7,468+ rows Truncated data 1.237s runtime Refreshed 5h ago

STEP 3 : Create a second dataset simulating new/incremental data.

CREATE INCREMENTAL DATASET

```
from pyspark.sql.functions import col
incremental_df = clean_df.limit(5) # using only 5 records
incremental_df = incremental_df.withColumn("Sales",col("Sales") + 100) # Modify Sales column to simulate data change
display(incremental_df)
```

> See performance (1)

> incremental_df: pyspark.sql.connect.dataframe.DataFrame = [Row ID: long, Order ID: string ... 19 more fields]

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	
1	423	CA-2017-125388	2017-10-19	2017-10-23	Standard Class	MP-17965	Michael Paige	Corporate	United States	Lawrence	Massachusetts
2	443	CA-2016-115756	2016-09-05	2016-09-07	Second Class	PK-19075	Pete Kriz	Consumer	United States	Detroit	Michigan
3	446	CA-2016-115756	2016-09-05	2016-09-07	Second Class	PK-19075	Pete Kriz	Consumer	United States	Detroit	Michigan
4	455	CA-2017-147277	2017-10-20	2017-10-24	Standard Class	EB-13705	Ed Braxton	Corporate	United States	Akron	Ohio
5	456	CA-2016-100153	2016-12-13	2016-12-17	Standard Class	KH-16630	Ken Heidel	Corporate	United States	Norman	Oklahoma

5 rows 0.857s runtime Refreshed 5h ago

12:21 PM (1s) 7 Python

```
from pyspark.sql import Row
# add new record
new_record = spark.createDataFrame([
    Row(
        **{
            "Row ID": 10001,
            "Order ID": "CA-46",
            "Order Date": "2025-08-08",
            "Ship Date": "2025-08-10",
            "Ship Mode": "Second Class",
            "Customer ID": "HD-87",
            "Customer Name": "Harshit",
            "Segment": "Consumer",
            "Country": "United States",
            "City": "New York",
            "State": "New York",
            "Postal Code": 101,
            "Region": "East",
            "Product ID": "CH-78",
            "Category": "Furniture",
            "Sub-Category": "Chairs",
            "Product Name": "Gaming Chair",
            "Sales": 1280.0,
            "Quantity": 2,
            "Discount": 0.0,
            "Profit": 250.0
        }
    )
])

# Add new record to incremental dataset
incremental_df = incremental_df.unionByName(new_record)
display(incremental_df)
```

> [See performance \(1\)](#) ✓ Checked for improvements

incremental_df: pyspark.sql.connect.dataframe.DataFrame = [Row ID: long, Order ID: string ... 19 more fields]
new_record: pyspark.sql.connect.dataframe.DataFrame = [Row ID: long, Order ID: string ... 19 more fields]

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	
1	944	CA-2015-120880	2015-05-29	2015-06-03	Standard Class	JL-15850	John Lucas	Consumer	United States	Seattle	Washingt
2	954	CA-2017-136539	2017-12-28	2018-01-01	Standard Class	GH-14665	Greg Hansen	Consumer	United States	Round Rock	Texas
3	1014	US-2015-126214	2015-12-21	2015-12-24	Second Class	JS-15880	John Stevenson	Consumer	United States	Seattle	Washingt
4	1042	CA-2016-102981	2016-09-06	2016-09-09	Second Class	MO-17500	Mary O'Rourke	Consumer	United States	New York City	New York
5	1053	US-2015-153500	2015-07-03	2015-07-05	First Class	DG-13300	Deirdre Greer	Corporate	United States	Philadelphia	Pennsylva
6	10001	CA-46	2025-08-08	2025-08-10	Second Class	HD-87	Harshit	Consumer	United States	New York	New York

6 rows 1.082s runtime Refreshed Now

STEP 4: Apply MERGE operation to update existing and insert new records.

05:16 PM (8s) 10

MERGE OPERATION

```
from delta.tables import DeltaTable

# Load Delta table
delta_table = DeltaTable.forName(spark, "workspace.default.sample_superstore")

# Perform MERGE operation
result = (delta_table.alias("target").merge(incremental_df.alias("source"), "target.'Row ID' = source.'Row ID'").whenMatchedUpdateAll().whenNotMatchedInsertAll().execute())
display(result)
```

> [See performance \(2\)](#) ✓ Checked for improvements

result: pyspark.sql.connect.dataframe.DataFrame

```
num_affected_rows: long
num_updated_rows: long
num_deleted_rows: long
num_inserted_rows: long
```

num_affected_rows	num_updated_rows	num_deleted_rows	num_inserted_rows
6	5	0	1

1 row 5.893s runtime Refreshed 34m ago

STEP 5: Validate results (row count, duplicates).

VALIDATION

05:16 PM (1s) 12

```
# Load updated Delta table
final_df = spark.table("workspace.default.sample_superstore")
display(final_df)
```

See performance (1) ✓ Checked for improvements

final_df: pyspark.sql.connect.dataframe.DataFrame = [Row ID: long, Order ID: string ... 19 more fields]

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida
6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California
7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California
9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California
10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California
11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California
12	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California
13	CA-2017-114412	2017-04-15	2017-04-20	Standard Class	AA-10480	Andrew Allen	Consumer	United States	Concord	North Carolina
14	CA-2016-161389	2016-12-05	2016-12-10	Standard Class	IM-15070	Irene Maddox	Consumer	United States	Seattle	Washington

7,465+ rows Truncated data 1.399s runtime Refreshed 38m ago

05:16 PM (<1s) 13

```
# validate row count
print("total number of rows :", final_df.count())
```

See performance (1) ✓ Checked for improvements

total number of rows : 9995

05:16 PM (1s) 14

```
# check for duplicates
duplicate_rows = final_df.count() - final_df.dropDuplicates().count()
print(" Total number of duplicate rows:", duplicate_rows)
```

See performance (2) ✓ Checked for improvements

Total number of duplicate rows: 52

05:16 PM (<1s) 15

```
# validate new record was successfully inserted
final_df.filter("Row ID = 10001").show()
```

See performance (1) ✓ Checked for improvements

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Size
10001	CA-46	2025-08-08	2025-08-10	Second Class	HD-87	Harshit	Consumer	United States	New York	New York	101	East	CH-78	Furniture	Chairs	Gaming Chair	12

STEP 6: Display final dataset

```
# display final output
display(final_df)
```

See performance (1)

Checked for improvements

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	Calif
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida
6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif
7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif
9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif
10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif
11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif
12	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Calif
13	CA-2017-114412	2017-04-15	2017-04-20	Standard Class	AA-10480	Andrew Allen	Consumer	United States	Concord	North
14	CA-2016-161389	2016-12-05	2016-12-10	Standard Class	IM-15070	Irene Maddox	Consumer	United States	Seattle	Washington
15	US-2015-118983	2015-11-22	2015-11-26	Standard Class	HP-14815	Harold Pawlan	Home Office	United States	Fort Worth	Texas
16	US-2015-118983	2015-11-22	2015-11-26	Standard Class	HP-14815	Harold Pawlan	Home Office	United States	Fort Worth	Texas

7,465+ rows Truncated data 1.246s runtime Refreshed 41m ago

```
# download incremental dataset
incremental_df.toPandas().to_csv(
    "customer_incremental.csv",
    index=False
)
```

05:16 PM (1s) 17

SUMMARY

This project shows incremental data processing using Delta Lake in Databricks. Firstly, the dataset was cleaned up by removing null values and duplicates, after that we have created an incremental dataset with updated records and insert a new record. A Delta Lake MERGE operation was performed to update existing records and insert new ones. At last the results were validated to verify successful incremental data processing and we downloaded the final incremental dataset.